

AI FACTORY · NATIVE.BUILDER HACKATHON 2026

Maya — CS Health Radar

A customer success manager owns 100 accounts.
She can actively manage 15. This agent handles the other 85.

`eonhw2qcm2ajxbmcp3jdtvopw.nativelyai.app`

THE PROBLEM

**Churn is decided in the accounts
nobody has time to look at.**

15 / **85**
managed per week left unwatched

The signals are already there — a P1 ticket, a usage collapse, a late invoice, a detractor survey. Nobody has 33 hours a week to read them.

WHY IT STAYS UNSOLVED

Three failures compound.

Signals are fragmented

Evidence sits in the support inbox, product analytics, the CRM and billing. Assembling one account's picture by hand takes ~20 minutes.

Detection is calendar-driven

Risk is discovered when a renewal date approaches. A problem with a five-month runway gets a 30-day response window.

Knowing is not acting

A credible, context-specific outreach takes 15–25 minutes to write. At 6pm the at-risk account gets a generic check-in — or nothing.

THE TARGET USER

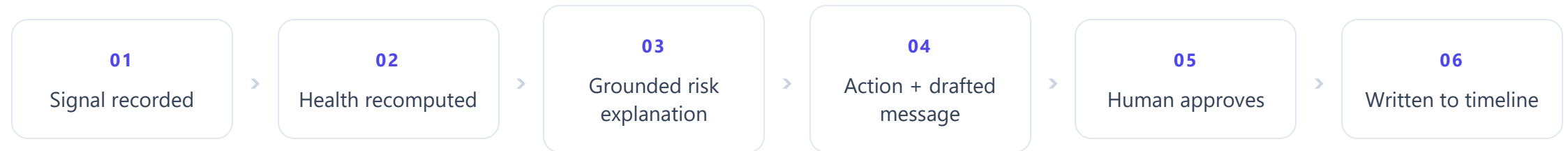
One person. Named, not generic.

An individual-contributor CSM at a growth-stage B2B SaaS company — 30–150 employees, \$3M–\$20M ARR. Accounts are \$8k–\$60k ACV: too small for a named account team, too large to ignore. She reports on Net Revenue Retention and is paid partly on renewals, so churn is personally expensive. She is fluent in SaaS tools, but will not write SQL and will not configure a rules engine.

Not for: enterprise CSMs with named account teams · support agents · VP-level portfolio reporting · PLG companies with no CSM function.

THE ONE WORKFLOW

Signal to logged human decision.



Every screen and every table in the product serves one of these six steps. Nothing else was built.

THE DECISION WORTH EXPLAINING

The score is code. Never a model.

```
// pure, synchronous, no network call, no AI
penalty(signal) = base_weight[type] × severity_mult[severity] × decay(ageDays)
decay(ageDays)  = max(0, 1 - ageDays / 90)
score           = clamp(round(100 - Σ penalty), 0, 100)
tier            = ≥70 healthy · ≥40 watch · else critical
```

The same signals always produce the same score — so cause and effect can be **verified live** rather than taken on trust. Every account page shows the full per-signal breakdown behind its number.

VERIFIED ON THE LIVE APP

Blue Ridge Partners — why 27?

SIGNAL	SEVERITY	AGE	PENALTY
Payment issue	critical	2.6 d	-34.01
Support ticket	high	4.6 d	-19.94
Usage drop	medium	8.6 d	-13.57
NPS response	low	12.6 d	-5.59
100 – 73.11			27

Not an opinion. Arithmetic a judge can check on screen.

Grounding is enforced in code and schema — not in the prompt.

- Every explanation must cite ≥ 2 **real signals** — a Postgres CHECK constraint makes an ungrounded recommendation impossible to save.
- Returned citation ids are validated server-side as a **subset of the context actually supplied**. An invented citation is rejected outright, not filtered out.
- Actions are bounded to a **six-value Postgres enum**. Out-of-taxonomy output retries once, then falls back to the driver default at low confidence.
- `signals` grants **SELECT and INSERT only**. Evidence is immutable — which is what makes the audit timeline trustworthy.

THE HUMAN BOUNDARY

Nothing is ever sent.

Approving records the decision and the final message text. It does not transmit anything.

There is **no email client, no SMTP configuration and no outbound sender anywhere in the codebase** — the guarantee is enforced by absence, not by a flag that could be flipped. The UI says so plainly, on every draft.

This is what makes it deployable in a real CS organisation on day one: the agent never touches a customer without a person signing off.

HOW IT WAS BUILT

native.builder, end to end.

Product Architect

Planned the project from a written brief — four screens, seven entities, a six-step workflow — and interrogated it with clarifying questions before any code was generated.

Builder agent

Generated the React front end, the full Postgres schema with row-level security, the seed dataset, and three Supabase Edge Functions calling Claude.

QA agent

Independently verified the build and caught two real defects — including a broken demo-user auth path silently returning zero rows through RLS.

Stack: Vite + React + Tailwind · Supabase (Postgres, Auth, Edge Functions, Secrets) · AI/ML API → anthropic/claude-opus-5 for the reasoning path, anthropic/claude-sonnet-5 for enrichment.

IMPACT

20 minutes → under 30 seconds.

That is the time to assess one account's risk: from opening four tools, reading recent history and forming a judgement, to a single ranked view with a grounded explanation already written.

The 20-minute baseline is stated as an assumption, not a measurement. We are **not** claiming a churn-reduction percentage — we have not run the experiment that would justify one.

Reproducible

Deterministic scoring. Same signals, same number, every time.

Auditable

Every score shows its per-signal breakdown. Every decision lands on a timeline.

Safe by construction

Human approval required. No send path exists in the code.

TRY IT

Maya — CS Health Radar

Ten seeded accounts. Open the top one and read why it is at risk.
Then add a signal to a healthy account and watch the score move.

<https://eonhw2qcm2ajxbmcp3jdtvopw.nativelyai.app>

Built with native.builder · Supabase · Claude via AI/ML API